Cryptography

Day 1: Encoding & Hashing Basics

Files path : davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1

1.Encode a string into Base64 and decode it back > 1.1.py

- Imported the built-in base64 module

- Took user input as a string

- Used .encode() to convert text into bytes

- Used base64.b64encode() to encode the string

- Used base64.b64decode() to decode it back to the original text

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.1.py
Enter text to encode: Hello
Encoded (Base64): SGVsbG8=
Decoded: Hello
```

2.Convert a string into its Hexadecimal representation > 1.2.py

- Took a text input from the user

- Converted the string to bytes using .encode()

- Used the .hex() method to convert the byte sequence into hexadecimal format

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.2.py
Enter text: Hello
Hexadecimal: 48656c6c6f
```

3.Write a script that calculates the MD5 hash of a given file > 1.3.py

- Imported the hashlib module for hashing operations

- Opened the file in binary mode (rb)

- Read the file contents and calculated its MD5 hash using hashlib.md5()

- Displayed the resulting hash in hexadecimal format using .hexdigest()

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.3.py
MD5 Hash: eb3d7bf02197ade0b780c321708e9d97
```

4.Generate the SHA-1 hash of a string > 1.4.py

- Imported the hashlib module for hashing operations

- Took a string input from the user

- Converted the string to bytes using .encode()

- Generated the SHA-1 hash using hashlib.sha1()

- Printed the resulting hash in hexadecimal format using .hexdigest()

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.4.py
Enter text: Hello
SHA1 Hash: f7ff9e8b7bb2e09b70935a5d785e0cc5d9d0abf0
```

5.Create a tool that takes a password and outputs its SHA-256 hash > 1.5.py

- Imported the hashlib module to perform hashing operations

- Took a password input from the user

- Converted the password into bytes using .encode()

- Generated the SHA-256 hash using hashlib.sha256()

- Printed the resulting hash using .hexdigest()

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.5.py
Enter password: admin
SHA256 Hash: 8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918
```

6.Verify if two files have the same SHA-256 hash (file integrity check) > 1.6.py

- Created a function to calculate the SHA-256 hash of a file

- Opened each file in binary mode (rb) and read its content

- Generated the SHA-256 hash using hashlib.sha256()

- Compared the hashes of both files to determine if they are identical

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.6.py
Enter first file path: file1.txt
Enter second file path: file2.txt
File 1 Hash: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
File 2 Hash: 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
Files are different

[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.6.py
Enter first file path: file1.txt
Enter second file path: file3.txt
File 1 Hash: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
File 2 Hash: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
Files are identical (Integrity verified)
```

7.Bruteforce simple MD5 hashes of 4-digit PINs (0000–9999) > 1.7.py

- Took a target PIN as input

- Used a loop to generate all possible 4-digit PIN combinations from 0000 to 9999

- Calculated the MD5 hash of each generated PIN using hashlib.md5()

- Compared the generated hash with the target hash

- If a match was found, the correct PIN was displayed

Screenshot:

```
PIN not found
PIN not found
PIN not found
PIN not found
PIN not found
PIN not found
PIN not found
PIN not found
PIN not found
PIN Found: 1234

[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.7.py
Enter 4-digit PIN (1234): 0001
PIN not found
PIN Found: 0001
```

8.Base64 encode a file (binary-safe) > 1.8.py

- Imported the base64 module for encoding operations

- Opened the file in binary mode (rb) and read its content

- Encoded the file data using base64.b64encode()

- Saved the encoded output to a new file

Screenshot:

```
C:\Users\BANANA\Downloads\Codes\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.8.py
File encoded successfully.
Output saved to: encoded_file.txt
```

9.Write a Python function that detects if a string is Base64 or Hex > 1.9.py

- Took an encoded string as input from the user

- Used bytes.fromhex() to test if the string is valid hexadecimal

- Used base64.b64decode() to test if the string is Base64 encoded

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.9.py
Enter encoded string: 48656c6c6f
This looks like HEX encoding

[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.9.py
Enter encoded string: SGVsbG8=
This looks like Base64 encoding
```

10.Build a script that hashes text using multiple algorithms (MD5, SHA1, SHA256) > 1.10.py

- Imported the hashlib module to generate multiple hashes

- Took a text input from the user and converted it to bytes using .encode()

- Generated MD5, SHA-1, and SHA-256 hashes for the same text

- Printed all hashes in hexadecimal format using .hexdigest() for comparison

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 1>python3 1.10.py
Enter text: Hello
MD5    : 8b1a9953c4611296a827abf8c47804d7
SHA1   : f7ff9e8b7bb2e09b70935a5d785e0cc5d9d0abf0
SHA256: 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

Day 2: Symmetric Encryption (AES, DES)

Files path : davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2

1.Encrypt and decrypt a message using AES-ECB mode > 2.1.py

- Imported AES from the Crypto.Cipher module and pad, unpad from Crypto.Util.Padding

- Defined a 16-byte AES key for encryption and decryption

- Took user input and converted it to bytes using .encode()

- Used pad() to make the plaintext size compatible with the AES block size

- Encrypted the plaintext using AES in ECB mode

- Decrypted the ciphertext and removed padding using unpad() to restore the original message

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.1.py
Enter message: hello
Encrypted: b' i\xd6\xa0\xe3\x0c\x93\xa1\xef\xcd\xf1!\xaaC\xb1\x94'
Decrypted: hello
```

2.Encrypt and decrypt using AES-CBC mode > 2.2.py

- Generated a random Initialization Vector (IV) using get_random_bytes(16)

- Created an AES cipher in CBC mode with a 16-byte key and IV

- Applied padding to the plaintext before encryption

- Encrypted the message and printed both ciphertext and IV

- Decrypted the ciphertext using the same key and IV and removed padding to recover the original message

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.2.py
Enter message: hello
Encrypted: b'\x82\xc0\x91\xa8N\x02\x93\xdf\xa1\x97\xe5<\xc6\xef|p'
IV: b'm\x1ff\x1be\xf1\x00MGD:\xb0\xb6C\xdc\xfe'
Decrypted: hello
```

3.Generate a random AES key (128/256 bit) using secrets or os.urandom > 2.3.py

- Used the secrets module to generate secure random AES keys

- Generated a 128-bit AES key using secrets.token_bytes(16)

- Generated a 256-bit AES key using secrets.token_bytes(32)

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.3.py
AES 128-bit key: b'\xc6\x02>E9\xa3\x94\x92\xf9\x1b/\x99\x9c\xca;7'
AES 256-bit key: b'g.\xab\xd6{*\xecP\x1bT\xf0\xd49\xbd\x17\xa1\x15@\x94.Y\x98\x98\xc85k|=|\xde\xe7v'
```

4.Write a script to pad and unpad plaintext (PKCS7 padding) > 2.4.py

- Imported pad and unpad from Crypto.Util.Padding

- Took user input and converted it to bytes

- Applied PKCS7 padding to the plaintext

- Printed the padded output

- Removed padding using unpad() and restored the original text

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.4.py
Enter text: hello
Padded: b'hello\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b'
Unpadded: hello
```

5.Encrypt and decrypt a file using AES > 2.5.py

- Opened the input file in binary mode and read its contents

- Encrypted the file data using AES-CBC mode with padding

- Saved the encrypted data into a new binary file

- Decrypted the encrypted file using the same key and IV

- Saved the decrypted output to verify file recovery

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.5.py
File encrypted successfully
b'\x9253\xc4\x022\xbf\xc1#T\no\xd5\x9f}B'
File decrypted successfully
b'Hello World!'
```

6.Decrypt an AES-encrypted file (provide correct key and IV) > 2.6.py

- Imported AES and unpad from Crypto library

- Provided the correct AES key and IV

- Opened the encrypted file in binary mode

- Decrypted the file using AES-CBC mode

- Removed padding from decrypted data

- Saved the original content into decrypted.txt

Screenshot:

```
(venv)[redacted]\Codes\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.6.py
File decrypted successfully
Decrypted content: Hello World!
```

7.Encrypt a string with DES encryption > 2.7.py

- Imported DES cipher from Crypto library

- Created an 8-byte DES key

- Took user input and converted it to bytes

- Applied PKCS7 padding

- Encrypted the plaintext using DES-ECB mode

- Decrypted the ciphertext and restored the original text

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.7.py
Enter text: hello
Encrypted: b'\xdf\x1ba,\x1a\x02\x8a\xad'
Decrypted: hello
```

8.Compare AES-ECB vs AES-CBC modes (encrypt same plaintext, observe difference) > 2.8.py

- Imported AES and padding utilities from Crypto library

- Used the same plaintext and key for both ECB and CBC modes

- Encrypted the plaintext using AES-ECB mode

- Encrypted the same plaintext using AES-CBC mode with a random IV

- Printed both ciphertexts to compare the results

- Observed that ECB produces repeated patterns while CBC produces different ciphertext blocks

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.8.py
ECB: b'7CxR\xd9\x9b6u_I\xd5\x04\xfc\xd9?\xaf'
CBC: b'\xbe;\xf3pLo\xa3\x13d\xae \xd5\xbe\xa0\x1f\xfa'
```

9.Build a mini tool that encrypts text with user-provided key and saves it to file > 2.9.py

- Imported AES, padding utilities, and random byte generator

- Took user input for plaintext and AES key

- Generated a random IV for CBC mode

- Encrypted the plaintext using AES-CBC

- Saved the IV and ciphertext into output.bin file

- Printed the IV for future decryption

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.9.py
Enter text: hello
Enter key (16 chars): 1234567890abcdef
Encrypted saved to output.bin
IV: b'\xf6Tb\xaal\x1c\x82\x18L\x0ez\x13\xf5\xa6N\xf8'
```

10.Implement AES encryption in CTR mode manually (counter based) > 2.10.py

- Imported AES and Counter from Crypto library

- Created a 16-byte AES key

- Took user input and converted it to bytes

- Created a counter object for CTR mode

- Encrypted the plaintext using AES-CTR mode

- Created a new counter for decryption

- Decrypted the ciphertext and restored the original text

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 2>python3 2.10.py
Enter text: hello
Encrypted: b'o/\x0f\x1d^'
Decrypted: hello
```

Day 3: Asymmetric Cryptography (RSA)

Files path : davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3

1.Generate an RSA private and public key pair in Python > 3.1.py

- Imported RSA from Crypto.PublicKey

- Generated a 2048-bit RSA key pair

- Exported and displayed the private key and public key

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.1.py
Private Key:
-----BEGIN RSA PRIVATE KEY-----
MIIEoQIBAAKCAQEAhkAQ5VgRIzczH1NyEq98wAGVHwu3gtiZ3LQDGCRc16wAu+Hg
fEOdRQrPGH1Z6BoxOSAuOlznrnWXjk39D3gbp/NNN2CS8XVIFY+QNPZbuy2E++og
v2BQQFBgn5snT+LYJswkiwQ8AM1fKX91LtOl77BcFIe6CVmcaKA+EhJAYbg5u9xk
D8ii5vn/aFPh/+QOtlaWP+pB5ZV0sZbsV9EpKGBDoxEgHw0dRvUnXcwooMWizR3p
2j6DquYkyKkavU7i9kyFfhOYJ8kxAYnPeGxKMeuTk2oQe0QvoW5BmFExAcAYKK9H
VIPcN4KYIi37s9jqCKWGqmWl8E6D5fp6ZPzn6QIDAQABAoH/DaJ0FNbc5ra0enn/
G+a2xMlkW8LU4HUfK0sdBVasFztg17QZGIXFcz2lytbwSqJeXW7nQwYsgYtv2QQS
r3+DT4tmTDmCswM2TvpFqxtaNahp2SvuJ4GgpOeE6LDZTUO8sDQLghLsBSjWgWBw
hlD4iV9f+rXOLyTqvbWImE1CPKpTihHMgwReCN8p28tpIh9nyLR7vCooYHqyNiVC
0NVBkdA9WH7flJWAW6+gdQfRb5psVvSaXm7cac4g/ZqCSP6UCL0CNnSJ/hgGW2Tp
V5r+auGZ6CBJuzX+tax1KwphZenFS49fO9YL8q5yQq9+IAUywQth0kPSrSJAdfVJ
6hjpAoGBALdVIJzE8k97E4N2EmLbcZ92yO5J4luOc9g4a/abUjzFnzVatK035Y8o
axkroTAAtQgfRHriBI7nV/g7O2FqWWbH6AkVn9lMzEKhC5bbYFkr4okFB35jVHxu
CnWve5S0sbSQRqTOSKqIP5kN4nsJMbFrga2B7+Stn9IYdmtlkf9DAoGBALt2hze4
9Q39PSX8g8TGlIPcAS15fxwVyEbnCMTi9F6TTMfK99SdunEmijQrshtqr4eMlRMp
GieB60wZ6M4cxZlwMtWBbFf8JCUDwIAhzY5WCNtpr2wc3rP7k2Eo2g4ix83aE0vw
fVsx8FhEOj2bwZNzXZWOZZCbuT+kEF8FdntjAoGBAKT53uoUNLUfXRT8yFjbfqjw
PKpqJF8dltOpGkaFyQBVGgrqQBJsbZBg39d7soY+7KmNcZNfFpj+R+RicPIdwKvE
EuMx6XGjMWB4CbTMXyx/WFDTDMZIXnvGjnUO4BeWbx+8gUXHOA71or15iCgNwwEU
bFKVzimrsFlgmBfXBAV/AoGAPz/en45oIDAxUX/BdcMKmPLB8vXrTS6MRF/vNX7d
tIs+fKqes74ECelW+ANGNLnqP3HPWoejq3DKUwLq6aALi5mvjRn91DHegB6j9Jdz
I3MjyeP+rBuRiGh5cW63I83YXDwYBqc9pGdVEHL0s5S522YxTSSeYAwqlK5mV6kg
MMECgYBxJIQsv11oh/MZdb7k2KOXxwgKJphq/d1+XxB0RV+Vnp0Ws8ZkvWkeQ1dV
Zezybfy6ShFOgj7sbR/NTWNRwSjkzbZN4gVSHiXk3Rv5qTYOwykYyTZWE2ufwPP9
L8d3WPqFVomiOmwgZUQvybV9R25CDFhjGCqnXckHD0YSpbwnSg==
-----END RSA PRIVATE KEY-----
Public Key:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAhkAQ5VgRIzczH1NyEq98
wAGVHwu3gtiZ3LQDGCRc16wAu+HgfEOdRQrPGH1Z6BoxOSAuOlznrnWXjk39D3gb
p/NNN2CS8XVIFY+QNPZbuy2E++ogv2BQQFBgn5snT+LYJswkiwQ8AM1fKX91LtOl
77BcFIe6CVmcaKA+EhJAYbg5u9xkD8ii5vn/aFPh/+QOtlaWP+pB5ZV0sZbsV9Ep
KGBDoxEgHw0dRvUnXcwooMWizR3p2j6DquYkyKkavU7i9kyFfhOYJ8kxAYnPeGxK
MeuTk2oQe0QvoW5BmFExAcAYKK9HVIPcN4KYIi37s9jqCKWGqmWl8E6D5fp6ZPzn
6QIDAQAB
-----END PUBLIC KEY-----
```

2.Encrypt a short message with RSA public key and decrypt with private key > 3.2.py

- Imported RSA and PKCS1_OAEP from Crypto

- Generated an RSA key pair

- Encrypted a byte string using the Public Key and PKCS1_OAEP padding

- Decrypted the resulting ciphertext using the Private Key to recover the original message

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.2.py
Encrypted: b'm\xea\x95\xa7\xa9\x88I\xf6\xdbh\x16\xe4o\x1c\x15 WU]\xd1\xb9\xc8\xe74\x87\xb5t\xfbU\x8a\x0cp/\xfdp\xe5\x82\x90\x93\x08\xc3\xd0\
xc6\xcc\xd82\x057F\xe3\xee\x9d\xfbC\xee\xf3\xf0m\xbd\x9f\x1ba\x1a\xd1\x8e\xc2\xf7`snzR\x0b\x1c\x1d{\xe8_\xcd\xbc\xa6r\x95\xa9&\xb8\x9e\x98\x
dd\x04\x1c\x04\x86L&% \xbcw\xfd\x9d\x02Gh\xf2\xe8\xc0\xde\x05\x88\x1e\xd8\xc5\xd7\xacY\xa9\xeb\xff\x19~&\x10\x8aj\x06r?S\x89\xa99\xea\xf8Yd\
xc6\xe4\xa6\xb7M\xf0\xc3\xfd\xc4\\\x0bU\xf8\xd2\x1d\x08\xf1H\xb0\x8fo3k\xc3\xcc\x1c\xf4\x05`\xc4\xa0\x13\xd5\x8d\xa7\xa6\x15\xf9#\xb9\xc8\xb
e!k8\x1b\x93}\xa7\xc4\xcf.\x87\xdf\x92Qm*\x0fGm|\x9f\xd1M\xe5\xe1\xb8[IH\xa8\xea \x1b\xaa\xe66\rQ\x11\xcd\xa3\xfbN/\x97,\xa1D\n\x8d/\x94!\x9
2\x9f\x1d\x7f?,8!\xd9/+^\x11\x95[\xa3\x90\xa1n\xd6\x95%\x83R\x85'
Decrypted: Hello World!
```

3. Write a script to sign a message with an RSA private key > 3.3.py

- Imported RSA, pkcs1_15, and SHA256

- Created a test message and hashed it using SHA-256

- Generated a Digital Signature by signing the hash with an RSA Private Key

- Printed the signature in hexadecimal format.

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.3.py
Message: Important system update file
Signature (Hex): 1b0bbb4d65387be11fba6cd33b2269814505ec40bc618c281f2819c98c2aaafccc228ec9f1e30a2686c1931b0df0439fbcabf39c3ee8a5e5949315baa9b
0458c2b3ead581e14d4bc47cd757741b9868504778cfc5e47fe4e6081a50156a2ed81e31d8a73048a27d0e1bb807fb130cdadf5913776cdf9ed8e77f8316dcc50cb45c66f4c0
67a6e6143ccd598245fe0d3a10c4670e66205b99ceca46404bf0d8324f82f9bb8746d79e207a0e2815e45464ec52ef89054b1860bd81f9f4d8d362d25f1b4aa5a4ac12516d33
fdcc453cb34ff051751eb99b51f31f3a841c081a36fcc6b535d73fbd39fc9a3b31baf34a64d8ebd26cd65fd4f9a553aae009fb3c1ff77
```

4. Verify an RSA signature with the public key > 3.4.py

- Imported RSA, pkcs1_15, and SHA256

- Created a hashed message and signed it to simulate receiving a signed file

- Hashed the received message

- Used the Public Key to verify the signature against the received hash

- Implemented try/except block to handle validation success or failure

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.4.py
Signature is valid.

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.4.py
Invalid signature! The message was tampered with or fake.
```

5. Save and load RSA keys from .pem files > 3.5.py

- Generated a 2048-bit RSA key pair

- Exported and saved the Private Key to a .pem file

- Exported and saved the Public Key to a .pem file

- Loaded both keys back into Python using RSA.import_key()

- Confirmed successful loading by checking the key size

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.5.py
Keys saved successfully to files
Keys loaded successfully from files
Loaded Private Key length: 2048 bits
```

6. Build a mini tool that encrypts text using RSA OAEP scheme > 3.6.py

- Imported a 2048-bit RSA key pair

- Built a command-line interface to take user input

- Encrypted the input using the RSA Public Key and PKCS1_OAEP padding

- Converted the raw ciphertext to Hexadecimal format for readable output

- Decrypted the text using the Private Key to verify data integrity

Screenshot:

```
(venv)[redacted]davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.6.py
--- RSA OAEP Encryption Tool ---
Enter text to encrypt: Hello

Encrypted (Hex):
889669cbeef99d05c61b20bb4a776ccd29efd7db6f018befccb6232819ac79498dcd110f3b0486ba1d37bb39b1ae6b83bf59ae52765b0926e1b481c3be5701949ced1848bae2
06182e2b9c1bc6ca69c0ffd6b3c4b354c1b744ef963450b3f4a641f76f34de02380b1121f9325062bc2c85a879708bdb92a00d266b59c1072f22a6e9311beda2ecb520a46c33
6301ff7702a6f20a06b4f1d337a11d46b9e7a244a8f3200832538195d80027702822b1a7ee45dd3c3e0968c8ca79010beabe57d5a85978f01ffdaf866f294ced5d48dfae0784
58421d1683fabbdf7bf9da6068a6d87abe010a9a022e402d439cac14543a76557c9c94e2bc470bd87605839d55cd

Decrypted:
Hello
```

7. Implement a hybrid encryption (RSA+AES): encrypt AES key with RSA > 3.7.py

- Imported a 2048-bit RSA key pair for the receiver

- Generated a random 16-byte AES key for the sender

- Encrypted a message using AES in CBC mode with padding

- Encrypted the AES key using the receiver's RSA public key with PKCS1_OAEP

- Decrypted the AES key using the receiver's RSA private key

- Decrypted the message using the recovered AES key and verified integrity

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.7.py
--- Sender ---
Encrypted AES Key (Hex): 8c8472a435a4cd7ac7cb29e96e0ee4d9683c4e1c72d3c0bbcae05b1bdc57327fc6e9b5d640a8f9c02f834035b8ae10111f1813990086e0bc3ea
a1656ea244a20f5c65b593600befe430021f44126dde8f525574209b08e7e22f2c4471155816a1394750fc147572f6982b29852985b79475e9a3e3765dfc80d1a237d381571b
7bccc794b079660d86b756159d71247de5f3562b749a682ecca10414c6a3499f592323d02512df436df3b42c23f2ea2bcd2da9984bb4d511523f902b6844303807c3ab72ab38
b25df588ee3145f664bfaa8647a6ecd99601b1bd0731b632cadd2a29efebaabc02f7cc9bd4c85ada67c0ad9228da0c2688e6c0022104ee4fc4610
Encrypted Data (Hex): d90b2b40e09a85454f0ef2f23e845049e28dce087d0706cf8837454cde71f3e7

--- Receiver ---
Decrypted Message: Top Secret Document
```

8. Write a script that encrypts large files using hybrid encryption > 3.8.py

- Loaded RSA keys from .pem files

- Encrypted the file contents using AES-CBC with a randomly generated session key

- Encrypted the AES session key using the RSA public key

- Combined and saved the encrypted AES key, IV, and ciphertext into a single binary file

- Read the binary file by extracting the exact byte lengths for the key and IV

- Successfully decrypted the session key and subsequently the file data, restoring the original document

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.8.py
File encrypted and saved to 'encrypted_file.bin'
Encrypted file content (hex): b'\xa6\x99L\xec\x13(j\xfa\x985\x0bi\x01\x18>J\x02\xd9\xab\x90\x9b3\xff\r\xc11\xe1\x8c\xc2\x9c\xa4\x00R\xc0\xfe
~\xef\rb\nm~\x0f\xaa\xbd\xae\xfb\x96\xe1R9**\xde\x8f\x9c\xac\x8fi&\xc2\xb7__J&\xa3\xf3\x0c\x96kgTN\xa1\xe3P\x0ej\x0c\xf1\x9c\xf0\xd4\xdfl\xa
d\xba\xbc\x94\xf0C2\xa0\xee\xd5\xd6\xe9\\\xda.3\x96\xdf\xf6\xd6\xb9\x89\x181\x04\xa80\xf7\xb9\xa2v|\x80\x13\xb9Tg\xdae\x8bh\xd4\x8f\xbeu \'\
xde\xda\xed\x01\x1f\x97\xc0f(\xd8tR\xba2\xdao\xe0\xd3a\xf04\xc4q\x96\x9e\xd0S\x9dR\x1e\xd0\x1dt`\x91]\xb7\xd0\xec~".B\xff9 \x8b\xf5Wl\x16\xc
d\xccq\xdcG\xa0%\xe9\x94\x1csq\x16\xa8\x13t\xc7\x0b\x06*\xcau\xdf\x88\xe8\xad\xd2S~^_D\xa6\x7f!\x06\xca\x99\xfbDr\x8b>\xa3\xcb\xbd\xbf\x86\x
c7i2\xc4RK\x85V\xe7v\xfb\xb4kg%\xbe#\x15j\x7f\xde\x14Q\xf03\x8f\x12*\xd0\xe0\x16uQ\xca\xa2H\xac\xe7\xfc\x0b\xb6\xdf\xa1\xe5`\x1c\xac\xc5\xed
\xf7\xeb\x95;?\xf8\xcb\x18\xe1`R\x81\xef\xf7\xceR\xaa7\xf72w\xfc\xab\xcb\xd0j\x08\xf9g\xa5 \x80\xe0\x9b\xba7\n\x13)'

File decrypted and saved to 'decrypted_document.txt'
Decrypted file content (hex): Sensitive Information Do Not Share Any Where
```

9. Crack a very small RSA keypair (for educational purposes) > 3.9.py

- Simulated an intercepted small RSA Public Key (n, e) and ciphertext

- Built a brute-force loop to factorize 'n' into its prime components 'p' and 'q' and Calculated Euler's totient function (phi)

- Used the modular inverse function to derive the private key 'd' from 'e' and 'phi'

- Decrypted the ciphertext manually using the RSA mathematical formula (pow) to recover the plaintext

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.9.py
Target Public Key -> n: 3233, e: 17
Found factors -> p: 53, q: 61
Cracked Private Key (d): 2753
Decrypted Message (Integer): 123
```

10. Write a script that tests RSA encryption/decryption speed for different key sizes > 3.10.py

- Failed to generate the 512-bit key because modern cryptography libraries (PyCryptodome) enforce a strict minimum of 1024 bits for security reasons, throwing a ValueError

- Documented the restriction and adjusted the script to test 1024, 2048, and 3072-bit keys

- Benchmarked key generation, encryption, and decryption speeds using the time module

- Observed the exponential increase in processing time as the key size grows

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_5_Cryptography\Day 3>python3 3.10.py

--- Testing 1024-bit RSA ---
Key Generation: 0.1051 seconds
Encryption: 0.000916 seconds
Decryption: 0.001364 seconds

--- Testing 2048-bit RSA ---
Key Generation: 0.7883 seconds
Encryption: 0.001128 seconds
Decryption: 0.004388 seconds

--- Testing 3072-bit RSA ---
Key Generation: 8.8137 seconds
Encryption: 0.002402 seconds
Decryption: 0.012567 seconds
```